MBAi 460
Modern Software Architectures
Spring 2026

Northwestern University

Lab Exercise #04 (v1.0)

Assignment: AWS Serverless computing with Lambda
Submission: Gradescope
Policy: Individual work (no teams), AI use encouraged
Complete By: Monday May 18th @ 11:59pm

Prerequisites: *class session Thursday May 7th on Lambda, Serverless computing, and API Gateway*

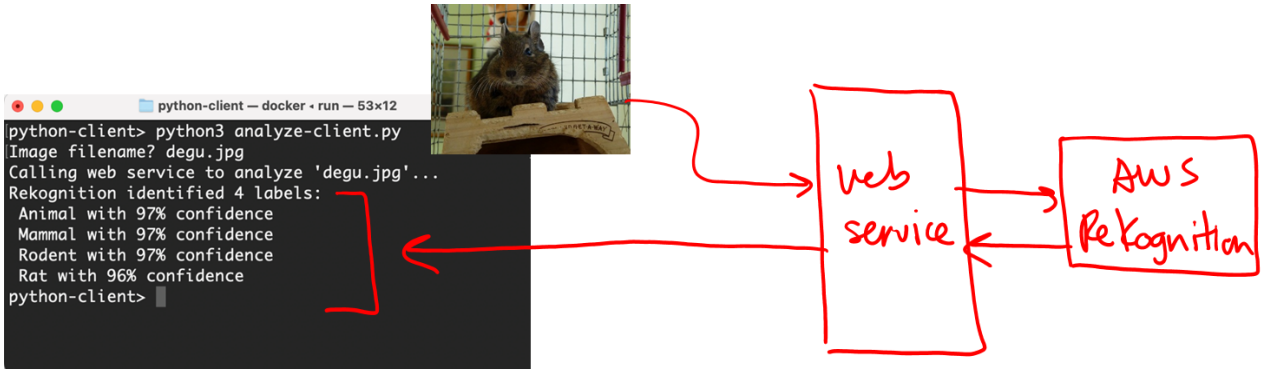
Overview

In project 02 we built a realistic multi-tier PhotoApp with our web service running in EC2, callable by clients anywhere in the world. What if AWS could build and deploy the web service for us, so all we had to do was write the code that interacted with MySQL, Rekognition, and S3? That's the idea behind ***Serverless computing with Lambda***. We focus on the code related to our application, let Amazon handle the rest.

Here in lab 04 you're going to build a small cloud app that performs two operations: (1) analyzes images with ***AWS Rekognition***, and (2) retrieve weather info from the free web service ***Open Meteo***. We are going to provide the design and implementation for #1, then you'll apply this design to implement #2.

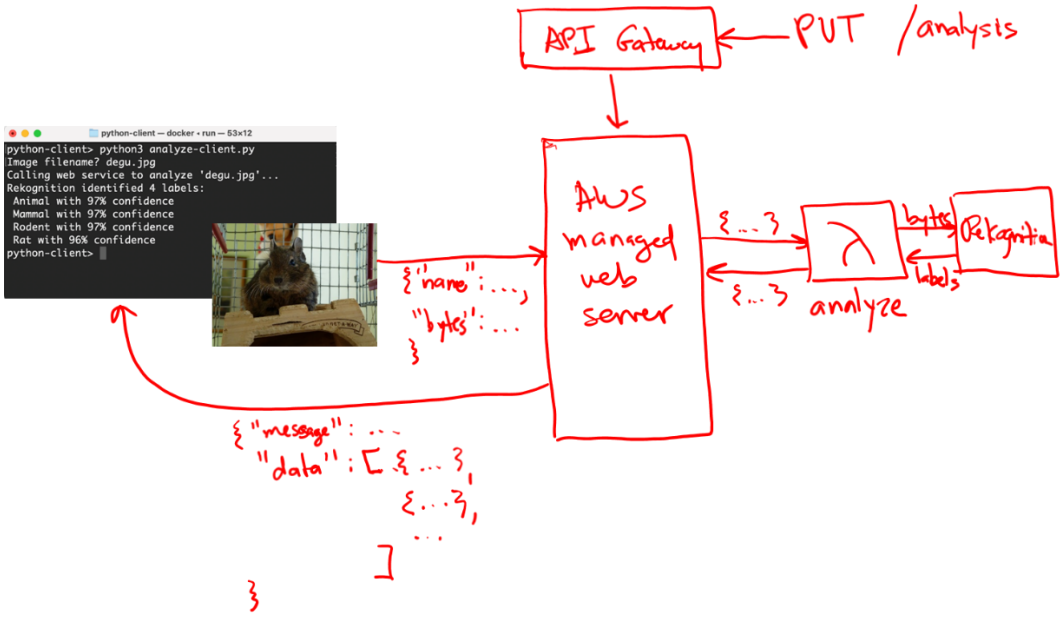
Operation #1: image analysis with AWS Rekognition

The initial goal is to build a web service that allows the client to upload a photo and have it analyzed by Amazon's ***Rekognition*** service. We saw this in projects 01 and 02:



The difference is that we are using a ***Serverless*** design where AWS builds the web service for us; ***API Gateway***

is the tool we use to configure the web service. The code for our web service function resides within a single Python-based ***Lambda*** function named ***analyze***, which is response for calling *Rekognition*:



The client issues a ***PUT*** request, sending the image name and bytes to the AWS managed web server (representing our web service). In response the web server calls the *analyze* Lambda function, which in turn calls the *Rekognition* service. The results from Rekognition are then sent back to the client, through the AWS managed web server, for display to the client. [***Note****: for simplicity we are NOT storing analysis results in DB*]

STEP 1: implement and test analyze Lambda function

Step 1 is to create and test the lambda function in Amazon's lambda console.

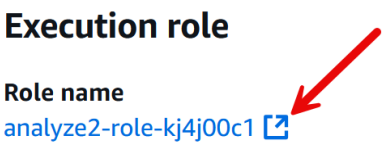
1. Open up AWS Lambda console.
2. Create a new function named "analyze", set "Runtime" to latest version of Python, click "Create".
3. Under configuration tab, click "Edit" and change the timeout to at least 5 minutes. Click "Save".
4. Under code tab, open "lambda_function.py" in the editor. Delete the given code.
5. In Dropbox, open "analyze.py", copy-paste into AWS Lambda editor.
6. Review the code… Notice how request body from client is obtained, call to Rekognition is made, and response from Rekognition is parsed and ultimately returned to client. As you know, exception handling is critical to ensure client receives a proper response in all cases.
7. Deploy
8. In Dropbox, open "test01.txt" and copy the contents.
9. In AWS Lambda console, switch to Test tab, create a new test, name it "test01", paste the text you copied from "test01.txt" into the Event JSON field (replacing what is there), and click "Save".

10. Now click the “Test” button --- it will take a few seconds to load and run, and then you can expand “Details” to see what happened. The response is shown first --- you’ll see that it failed with status code 500, “… not authorized to perform: rekognition:DetectLabels”. Scroll down further, and you’ll see the output from the print( ) statements in the lambda code:

    START RequestId: 7903c74a-e431-4f2f-9064-036015f74d26 Version: $LATEST
    ***Call to analyze...*
    ***Accessing request body*
    name: gray.jpg
    bytes (first 32 chars): iVBORw0KGgoAAAANSUhEUgAAAAEAAAAB
    ***Calling Rekognition*
    ***Exception*
    ***Message: An error occurred (AccessDeniedException) when calling the DetectLabels operation: User: arn:aws:sts::444800541605:assumed-role/analyze2-role-kj4j00c1/analyze2 is not authorized to perform: rekognition:DetectLabels because no identity-based policy allows the rekognition:DetectLabels action*

11. FYI, print statements in Lambda functions are the primary mechanism for debugging in the cloud. The AWS Lambda console contains a “Monitor” tab where Amazon collects all print( ) output in the form of “CloudWatch logs”; checking these logs is the first step in debugging cloud-based apps.

12. Back to the error… The lambda function does not have permission to access the Rekognition system. In the AWS console, click on the “Configuration” tab. Then along the left menu, click “Permissions”. At the top of the window pane you’ll see “Execution Role” with a Role name --- click the little box with arrow to the right of the name…



13. This will take you to IAM, Amazon’s **Identity and Access Management** console for this “role”, which is basically the permissions that your lambda function has when it executes. Click “Add permissions” drop-down, and select “Attach policies”.

14. In the search box, enter “Rekognition”. In the list that appears, check the box for “AmazonRekognitionFullAccess” and click “Add permissions”. The policy will be added to the role, and now the lamba function will have permission to call Rekognition.

15. Reopen the AWS Lambda console, and click on your analyze function to reopen --- if you don’t see it listed, check the Region drop-down in the title bar and make sure it says “United States (Ohio)”.

16. Switch to the “Test” tab, and click the “Test” button. Now under details you should see a response with status code 200 and one label back from Rekognition:

```
{
  "statusCode": 200,
  "body": "{\"message\": \"success\", \"data\": [{\"label\": \"Gray\", \"confidence\": 90}]}"
}
```

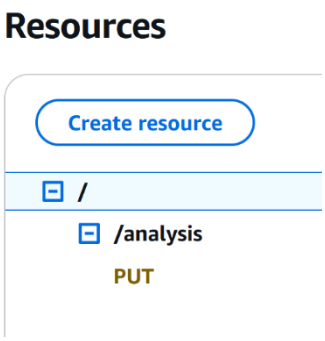
    Important implementation detail: look at the body of the response --- if you look carefully, notice that it’s one big string, with \” used to escape sub-strings. This is what JSON looks like, and confirms that the lambda function has responded correctly in a platform-neutral way. Also, scroll down and look at the output from the print( ) statements to confirm the lambda function executed successfully.

17. Your lambda function is now ready! If you want, there’s a 2nd file in Dropbox “test02.txt” file if you want to add and run a second test case. Test02 should return one label “Pattern” with 97% confidence.

STEP 2: create Web Service API

Step 2 is to create the web service to allow client access to the analyze function. This is the serverless component, where we use Amazon's **API Gateway** to define and deploy our API, in which case Amazon allocates and configures a web server for our web service. Right now the API will define only one entry point, **PUT /analysis**, which you'll map to the analyze function. The API can easily be extended if we add more functionality. [*This might be a good time to review the design diagram at the top of page 2?*]

1. In AWS console, search for AWS API Gateway.
2. Click "Create API" to get started.
3. Scroll down to "REST API", and click "Build".
4. Name the API "ServicesAPI" and click "Create API".
5. Create a resource --- think *function entry point* --- by clicking "Create Resource". Under Resource name, enter "analysis". Click "Create resource".
6. The resource "/analysis" should be selected in the UI. Click "Create method". Under Method type select the HTTP verb **PUT**. Lambda function should be selected. Scroll down, and enable "Lambda proxy integration" by sliding over the button. Then under Lambda function, click in the text field to drop-down a list of your lambda functions, and select "function:analyze". Scroll down and click "Create method".



7. Your API should look like this ---------->
8. Click the "Deploy API" button to have AWS allocate and configure our web service. Under Stage you'll be asked to choose an option --- mimic what companies do and create a new stage named "test" or "prod" (for production). Companies do this so that a single web service can host multiple versions of their web service, e.g. the current production version and a newer test version.
9. Click "Deploy" to start the deployment process.
10. Scroll down and copy the URL denoting your web service:

11. Create and open a text file and paste this URL for future reference. The web service is now ready to use!

STEP 3: client-side app

Step 3 is to write the client-side code to analyze images by calling the web service. A working solution is provided, all you need to do is change the base URL to denote your web service endpoint.

1. If you are working with the provided docker images, open a Terminal / PowerShell window in your local "mbai460-client-main" folder. Run docker with "./docker/run", cd into **labs**, and cd into **lab04**. Use **ls** to list the contents:

```
hummel> ./docker/run
client:~$ cd labs
client:~/labs$ cd lab04
client:~/labs/lab04$ ls
analyze.py    client.py  earth.jpg      test01.txt
chicago.jpg   degu.jpg   no-labels.jpg  test02.txt
client:~/labs/lab04$
```

   If you are not using the client docker image, you can find these files in Dropbox.

2. Open "client.py" in VS Code / your preferred editor. Change "baseurl" to denote YOUR web service endpoint.

3. Review the code, which should be very familiar to you now… The client's job is to obtain the name of the image file from the user, read the contents of that file as raw bytes, and then base64-encode for transmission. The data packet is constructed, and passed in the body of the requests via the PUT call. The data packet is sent in JSON format:

```
response = requests.put(url, json=data)
```

   After the call returns, the status_code is checked. If successful (200), the labels are extracted from the body of the response, and output one by one.

4. Run the client-side app: **python3 client.py**

5. Enter an image name: chicago.jpg, degu.jpg, earth.jpg, and no-labels.jpg. Here's an example run with "degu.jpg":

```
codio@julylibra-violinconduct:~/workspace$ python3 client.py
Image filename? degu.jpg
Calling web service to analyze 'degu.jpg'...
Rekognition identified 4 labels:
 Animal with 97% confidence
 Mammal with 97% confidence
 Rodent with 97% confidence
 Rat with 96% confidence
codio@julylibra-violinconduct:~/workspace$
```

6. How to debug lambda functions since they are running in the cloud? Open the AWS Lambda console, and open your *analyze* function. Click on the "Monitor" tab, and click on "View Cloudwatch logs". AWS saves the output of every call, including the output from the print( ) statements. You should see the output from the most recent call to analyze "degu.jpg".

7. Well done!

Operation #2: current weather with Open Meteo

The second goal is to implement a web service function that retrieves the current weather for a given city. We are going to use the free, no-token-required service https://open-meteo.com/, which provides an API for retrieving detailed weather information. Given a city, we'll use this service to first translate the city into a (latitude, longitude) position, then obtain the current weather at that position. For example, suppose we want to lookup the current weather in Chicago. First we obtain the (latitude, longitude) for Chicago:

https://geocoding-api.open-meteo.com/v1/search?name=Chicago&count=1&countryCode=US

Go ahead and execute this URL in a browser and see what you get back. Given the position, we can then get the current weather:

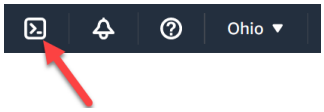
https://api.open-meteo.com/v1/forecast?latitude=41.85&longitude=-87.65&current_weather=true

Execute the above URL… Notice the temperature is in degrees Celsius, and the wind speed in KMH; you'll parameterize the URL to yield degrees Fahrenheit and MPH. Your web service function will be called via **GET /weather/country/city** where the two-letter country and city are passed as URL parameters. If the city is more than one word (e.g. "San Jose"), the space(s) are replaced with %20 (e.g. **GET /weather/US/San%20Jose**).

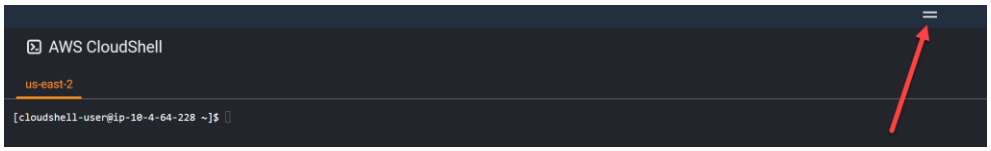
STEP 1: package Python requests library as a Lambda Layer

To call the Open Meteo web service we're going to use Python's ***requests*** library; since *requests* is not part of the standard Python installation, we'll have to package it up and make it available to the lambda function. These "packages" are called "layers" in Lambda terminology. We're going to use AWS's integrated ***CloudShell*** (aka Linux environment) to build the necessary layer and deploy to S3.

1. Open AWS Cloudshell via small button above your management console



2. A Linux-based command-line shell will open at the bottom --- you may have to grab the slider to enlarge the window so you can see it:



3. Play around for a minute --- "ls", etc. It's Linux.
4. The goal here is to create a .zip file that contains the necessary ***requests*** library. This is created as follows (be very careful to type this exactly as shown, e.g. notice the "." at the end of the 3rd command):

```
mkdir python
cd python
pip3 install requests typing_extensions==4.14.1 -t .
cd ..
zip -r requests-layer.zip python
ls
```

You'll get an error about "*pip's dependency resolver…*". You can safely ignore this error.

5. The “ls” command should reveal at least two items: “requests-layer.zip” and “python”. Success!
6. To make the .zip available to Lambda, we need to move the .zip file over to S3. Decide which bucket you want to use for storing the .zip file --- you can use any bucket you want, or create a new bucket for lambda-related files. [*Recall you can create a new bucket using CLI: aws s3 mb s3://bucket-name*]
7. Copy the .zip to your S3 bucket using the AWS command-line interface (CLI):

```
aws s3 cp requests-layer.zip s3://your-bucket-name
```

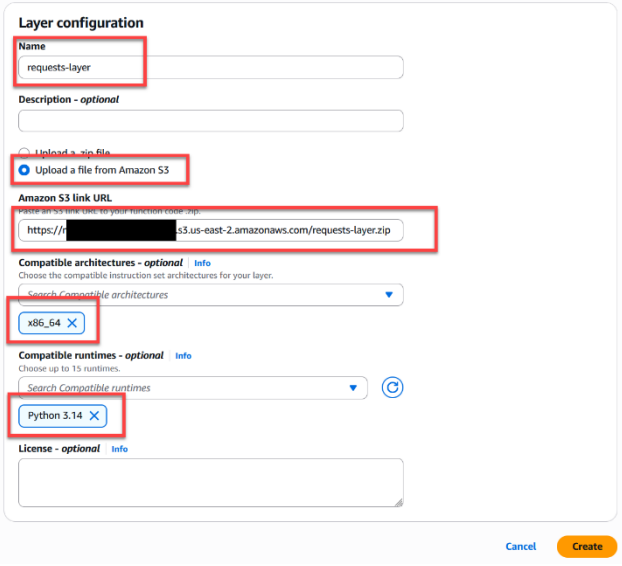
8. View your S3 bucket and confirm the .zip is there. Click on the .zip file, and in the window that opens, make a copy of the “**object URL**” --- it should look something like https://bucket-name.s3.us-east-2.amazonaws.com/request-layers.zip
9. Assuming the .zip was built correctly, you’re done --- close the CloudShell window.

The next step is to install the .zip into your AWS Lambda infrastructure. Move to the “top” of the Lambda service window:

Now click “Layers” and then the “Create layer” button:

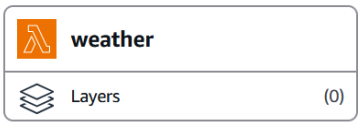
A window opens for you to configure the layer. Complete the highlighted areas --- note that you’ll paste the “object URL” that you saved earlier to refer to the .zip file as shown to the right.

When you’re ready, click the “Create” button to create the layer and install it into AWS’s Lambda infrastructure for use.

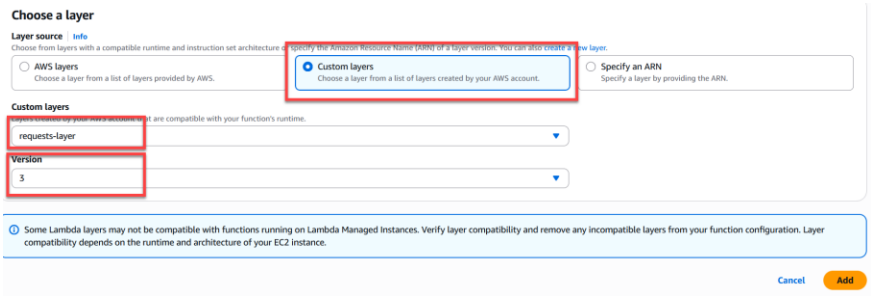


STEP 2: create and test your weather Lambda function

Create a new lambda function to perform the weather operation. After you create the function, use the Configuration tab to set the timeout to at least 5 minutes. Next, attach the layer you just installed to your new lambda function as follows:



1. At the top of your function under "Function overview", you'll see Layers.
2. Click on "Layers", and then "Add a layer".
3. As shown below, click on "Custom layers", and then select your requests layer.
   Be sure to also pick a version from the drop-down --- your version will most likely be 1.



4. Click "Add" and the requests library is now available to your lambda function.

At this point you are ready to start programming your weather function. There are 4 main steps you need to perform:

1. Obtain country and city parameters
2. Call Open Meteo API to obtain position of city
3. Call Open Meteo API to obtain current weather at this point
4. Return appropriate response

Use the provided code for the ***analyze*** lambda function as a guide. The Open Meteo *geocoding* API for obtaining city positions is documented @ https://open-meteo.com/en/docs/geocoding-api; the weather API is documented @ https://open-meteo.com/en/docs. Skim through the API documentation to see what parameters are available, and be sure to ***scroll down to the bottom*** to see examples of the format for response data / errors. Here are some notes as you implement the function and call the APIs:

1. Debugging? Use Python print( ) statements, which can be viewed in the output when you run a test.
2. The parameters to your web service function are URL path parameters, so you obtain via event["pathParameters"]. Test cases in the Lambda console will have this format:

```
{
  "pathParameters": {
     "country": "us",
     "city": "san%20jose"
  }
}
```

3. How do you call the Open Meteo API? In this case you are the client and Open Meteo is the web service, so call the API functions as you normally would from a **Python client**. Check the response status code, deserialize the body of the response since it will be in JSON format, etc. Any non-200 status codes from

the Open Meteo API should be reported back as an error message; see the API docs for how to obtain Open Meteo's error message.

4. The country code must be in UPPERCASE when passed to the Open Meteo geocoding API. Convert the ***country*** parameter to upper-case to make sure it will be accepted by the API. The case of the city does not appear to matter, so you can leave as is. Do not try to validate the country code or the city, leave that to the APIs you're calling. [**NOTE***: if the country code or city cannot be found, the geocoding API still returns status code 200, but without any data. To detect this "client-side error in calling the service", you have to deserialize the body of the response and check to see if "results" not in body, in which case your web service function returns a status code 400 with the error message "no such country or city"*]

5. Round the latitude and longitude to 2 decimal places before passing to the weather API function; use Python's ***round(value, 2)*** function.

6. We want the temperature in Fahrenheit and wind speed in MPH. These are controlled by additional parameters in the API call: *temperature_unit=fahrenheit* and *wind_speed_unit=mph*.

7. When your lambda function is ready to respond back to the client, the body of the response should be formatted as follows:

```
body = {
  "message": string,
  "temperature": number,
  "windspeed": number,
  "winddirection": integer,
  "synopsis": string
}
```

   The data must be sent in JSON format. On success the message should be "success" and the other keys filled with values. On an error, message should contain the error message and the other keys are undefined (i.e. any value may be returned). The status code should reflect proper HTTP errors code: 200 => success, 400 => client-side error, and 500 => server-side error.

8. The "synopsis" is a short textual description of the weather. The API returns a weather code, which you can map to a synopsis using the following Python dictionary (feel free to copy-paste this code):

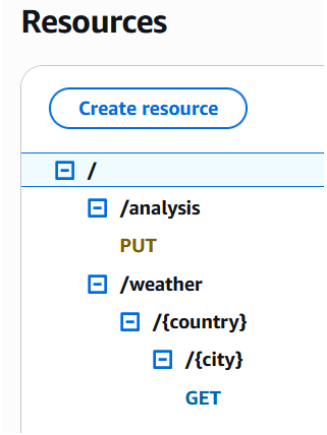
```
weather    = {}
weather[0] = "Clear sky"
weather[1] = "Mainly clear"
weather[2] = "Partly cloudy"
weather[3] = "Overcast"
weather[45] = "Fog"
weather[48] = "Fog"
weather[51] = "Drizzle: light"
weather[53] = "Drizzle: moderate"
weather[55] = "Drizzle: dense"
weather[56] = "Freezing drizzle: light"
weather[57] = "Freezing drizzle: dense"
weather[61] = "Rain: slight"
weather[63] = "Rain: moderate"
weather[65] = "Rain: heavy"
weather[66] = "Freezing rain: light"
weather[67] = "Freezing rain: heavy"
weather[71] = "Snow fall: slight"
weather[73] = "Snow fall: moderate"
```

```
weather[75] = "Snow fall: heavy"
weather[77] = "Snow grains"
weather[80] = "Rain showers: slight"
weather[81] = "Rain showers: moderate"
weather[82] = "Rain showers: violent"
weather[85] = "Snow showers: slight"
weather[86] = "Snow showers: heavy"
weather[95] = "Thunderstorm"
weather[96] = "Thunderstorm with slight hail"
weather[99] = "Thunderstorm with heavy hail"
```

Check to make sure the weather code exists in the dictionary. If the code cannot be found, return the following string as the synopsis:

```
synopsis = f"unknown weather code {weathercode}"
```

STEP 3: integrate with web service and test from client



Once the lambda function is working, the final step is to integrate the lambda function into the web service using API Gateway. Follow the steps shown earlier to edit and extend the "ServicesAPI" API Gateway you created earlier. Your lambda function is called via **GET /weather/country/city**, and be careful not to miss any configuration steps (in particular don't forget to enable "Lambda proxy integration"). Also remember that "country" and "city" are parameters, so you specify their resources in API Gateway using ***{country}*** and ***{city}***. Re-deploy your new API Gateway, and test from the client.

Grading and Electronic Submission

Your work will be collected and graded using Gradescope. We are grading your AWS deployment (not your client-side testing code), so all we need is your web service endpoint. Create a config file named "**lab04-client-config.ini**" with the following format:

```
[client]
webservice=https://...us-east-2.amazonaws.com/prod
```

The URL should start with https://, and should end WITHOUT a /. If you created a stage (e.g. "test" or "prod"), your URL should end with the name of that stage. If you are working in Docker, you can submit your config file as follows:

```
/gradescope/gs submit 1288073 8138322 lab04-client-config.ini
```

Your goal is an autograder score of 100/100; you have unlimited submissions. If the above fails, then gradescope is not yet accepting submissions; come back and try again later. If you are working outside Docker,

you can submit from your LOCAL computer by viewing your folder locally using Mac/Windows/Linux, open Gradescope site “Lab 04 – lambda” and drag-drop your .ini file.

By default, Gradescope records the score of your **LAST SUBMISSION** (not your highest submission score). If you want us to grade a different submission, activate the appropriate submission via your “Submission History”. This must be done before the due date.